

DSA MINI PROJECT

Expression Calculator

Infix-to-Postfix Conversion & Stack-Based Evaluation in C

Concepts: Stacks • Shunting-Yard Algorithm • Postfix Evaluation

Author: <Your Name Here>

TOP



Why Do We Need This Project?

The Problem

Humans write expressions like **3 + 4 * 2** (infix notation). A computer cannot evaluate this directly in one left-to-right pass, because it must respect **operator precedence** and **parentheses**.

The Solution

Convert the expression to **postfix (Reverse Polish) notation** first, using a stack. A postfix expression can then be evaluated in a single, simple left-to-right scan — no precedence rules needed at evaluation time.

INFIX

3 + 4 * 2 / (1 - 5) ^ 2



shunting-yard algorithm

POSTFIX

3 4 2 * 1 5 - 2 ^ / +

Result: 2.75

Core Concepts & Program Flow

Stack (LIFO)

CharStack (operators) & NumStack (doubles) — array-based, MAX = 200.

Infix Notation

Operator between operands: $A + B * C$. Needs precedence & parentheses rules.

Postfix Notation

Operator after operands: $A B C * +$. Evaluated directly, left to right.

Program Flow



Menu-driven UI: Evaluate • Postfix Only • Rules • Exit. Every stage detects empty input, unbalanced parentheses, invalid characters, and division/modulo by zero — the program never crashes on bad input.

Conversion & Evaluation Logic

Infix → Postfix (Shunting-Yard)

Numbers / decimals — appended straight to the output.

Operator seen — pop higher/equal-precedence operators (per associativity), then push it.

'(' / ')' seen — push '(' directly; on ')', pop to output until the matching '(' and discard it.

Unary +/- & end of input — insert an implicit 0 for unary signs; at the end, pop all remaining operators.

Postfix Evaluation

Number token — convert with `atof()` and push onto the stack.

Operator token — pop two values, apply `applyOp(a, b, op)`, push the result back.

End of string — exactly one value should remain — that is the final result.

PRECEDENCE TABLE

$\wedge$	level 3 Right-associative
$*$ $/$ $\%$	level 2 Left-associative
$+$ $-$	level 1 Left-associative
unary +/-	Rewritten as $0 - x$

Higher number = evaluated first

Example Run & Key Features

Infix: $3 + 4 * 2 / (1 - 5) ^ 2$



Postfix: $3\ 4\ 2\ *\ 1\ 5\ -\ 2\ ^\ /\ +$



Evaluation Steps

$4 * 2 = 8 \rightarrow 1 - 5 = -4, (-4)^2 = 16 \rightarrow 8 / 16 = 0.5 \rightarrow 3 + 0.5 = 2.75$

FINAL RESULT

`printf("%.6g", result) - 6 significant digits`

2.75

Key Features

- 1 Interactive Menu** — 4 options: evaluate, postfix only, rules, exit.
- 2 Unary Operator Support** — Handles $-5 + 3$, $4 * -2$ via implicit 0.
- 3 Right-Associative '^'** — Power evaluates right-to-left correctly.
- 4 Full Error Handling** — Empty input, brackets, invalid chars, div/mod by 0.
- 5 Modular Design** — Reusable CharStack & NumStack ADTs.
- 6 Readable Output** — Space-separated postfix tokens.

Error Handling & Conclusion

Issue	Example	How the code responds
Empty / unbalanced input	blank input, or "(3 + 4"	isBalanced() catches it upfront; re-prompts
Invalid char / bad brackets	stray '@', or extra '))'	infixToPostfix() returns failure
Division / modulo by zero	e.g. 5 / 0	applyOp() flags error; evaluation stops
Malformed postfix	too many / few operands	evaluatePostfix() checks final stack size

Project Summary

- Two custom stack ADTs (char & double) in C
- Shunting-Yard algorithm for infix $\rightarrow$ postfix
- Postfix evaluation with a numeric stack
- Robust validation & error handling throughout

Future Scope

- Support variables & constants (x, pi)
- Add functions: sin, cos, sqrt, log
- Dynamic memory instead of fixed MAX
- GUI front-end + expression history